# Mutabilidad 🧱

En `Python`, cada tipo de dato que usamos tiene una propiedad muy importante, para algunos podemos modificar su contenido después de ser creados, y otros deben permanecer fijos después de ser creados.

A esta característica se le llama `mutabilidad`, y entenderla nos permite escribir código más predecible, más seguro y más eficiente.

Pensemos en una construcción de **LEGO**. Empezamos con una base, vamos agregando piezas, quitamos algunas, reemplazamos otras. La construcción que vemos al final no es la que teníamos al inicio, es el resultado de todas las decisiones que tomamos en el camino. El valor cambió, pero siempre fue *la misma construcción* en el mismo lugar.

---

## ¿Por qué existe esta diferencia? 🤔

Podríamos preguntarnos: ¿por qué no hacer que todo sea `mutable`, o que todo sea `inmutable`? De esta manera sería más fácil su manejo, ¿cierto? La respuesta está en la naturaleza de los datos que manejamos en un programa.

Algunos datos representan un estado que `evoluciona`, como la lista de productos en un carrito, los mensajes de un chat, los resultados que vamos acumulando. Otros representan `hechos fijos` como las coordenadas de un punto en un mapa, el nombre de usuario en una sesión, las claves de configuración de una aplicación.

Si todo fuera `mutable`, el código podría modificar cualquier dato en cualquier momento, incluso sin que lo sepamos. Esto genera **efectos secundarios impredecibles** y bugs difíciles de rastrear.

Si todo fuera `inmutable`, cada pequeña modificación requeriría crear un objeto completamente nuevo. Para datos que cambian con frecuencia y que pueden ser grandes, esto sería costoso en memoria y complicado de manejar.

💡 `Python` nos da ambos tipos para que podamos **expresar nuestra intención como programadores**: *"este dato va a cambiar durante la ejecución"* vs *"este dato es un hecho permanente que no debería alterarse"*.

---

## Objetos mutables 🔧

Un `objeto mutable` puede modificarse directamente después de haber sido creado. `Python` mantiene el mismo objeto en memoria y actualiza su contenido sin necesidad de reemplazarlo.

A continuación veamos los tipos de dato `mutables` dentro de `Python`:

---

### Listas 📋

Las `list` son colecciones ordenadas y modificables. Podemos reemplazar un elemento, agregar uno nuevo al final, o eliminar cualquiera. En este ejemplo podemos ver esos procesos:

Copiar

```python
tasks = ["buy groceries", "send email", "call doctor"]
print("Original:", tasks)

# Reemplazar un elemento
tasks[1] = "send report"
print("Despues de reemplazar:", tasks)

# Agregar un nuevo elemento
tasks.append("read book")
print("Despues de agregar:", tasks)

# Eliminar un elemento
tasks.remove("call doctor")
print("Despues de eliminar:", tasks)
```

Original: ['buy groceries', 'send email', 'call doctor']
Despues de reemplazar: ['buy groceries', 'send report', 'call doctor']
Despues de agregar: ['buy groceries', 'send report', 'call doctor', 'read book']
Despues de eliminar: ['buy groceries', 'send report', 'read book']

---

## Diccionarios 🗂️

Los `dict` son colecciones de pares clave-valor. Podemos modificar el valor de una clave existente, agregar nuevas entradas o eliminar las que ya no necesitamos. Por ejemplo:

```python
user = {"name": "Maria", "age": 28, "city": "Buenos Aires"}
print("Original:", user)

# Modificar un valor existente
user["age"] = 29
print("Despues de modificar:", user)

# Eliminar un par clave-valor
del user["city"]
print("Despues de eliminar:", user)
```

Copiar

Original: {'name': 'Maria', 'age': 28, 'city': 'Buenos Aires'}
Despues de modificar: {'name': 'Maria', 'age': 29, 'city': 'Buenos Aires'}
Despues de eliminar: {'name': 'Maria', 'age': 29, 'email': 'maria@example.com'}

---

## Conjuntos 🔵

Los `set` son colecciones de elementos únicos y sin orden. Podemos agregar o eliminar elementos del conjunto directamente.

```python
active_users = {"alice", "bob", "carlos"}
print("Original:", active_users)

# Agregar un elemento
active_users.add("diana")
print("Despues de agregar:", active_users)

# Eliminar un elemento
active_users.remove("bob")
print("Despues de eliminar:", active_users)
```

Copiar

Original: {'alice', 'bob', 'carlos'}
Despues de agregar: {'alice', 'bob', 'carlos', 'diana'}
Despues de eliminar: {'alice', 'carlos', 'diana'}

---

## Objetos inmutables 🔒

Un `objeto inmutable` no puede modificarse después de haber sido creado. La única forma de obtener una versión modificada es crear un objeto nuevo con el contenido deseado.

Al igual que una pieza de cerámica, mientras la arcilla está blanda, podemos moldearla, agregarle detalles, darle la forma que queramos. Pero una vez que entra al horno y se hornea, su forma queda fija para siempre. Si necesitamos algo diferente, debemos crear una pieza nueva desde cero.

---

## Tuplas 📦

Las `tuple` son colecciones ordenadas cuyo contenido no puede modificarse. Son ideales para representar grupos de valores que deben permanecer juntos e inalterables.

```
1  coordinates = (40.7128, -74.0060)  # Coordenadas de Nueva York
2  print("Coordenadas:", coordinates)
3
4  # Intentar modificar un valor genera un error
5  coordinates[0] = 0.0
```

Copiar

✅ Coordenadas: (40.7128, -74.006)

❌ TypeError: 'tuple' object does not support item assignment

---

## Cadenas de texto 💬

Los `str` son secuencias de caracteres inmutables. No podemos modificar un carácter en su posición. Cualquier operación sobre un `str` produce un `str` nuevo, dejando el original intacto.

```python
greeting = "hello"
print("Original:", greeting)

# Para obtener una version modificada, creamos un string nuevo
capitalized = greeting.capitalize()
print("String nuevo:", capitalized)
print("Original sin cambios:", greeting)

# Intentar modificar un caracter genera un error
greeting[0] = "H"
```

Copiar

Original: hello
String nuevo: Hello
Original sin cambios: hello

TypeError: 'str' object does not support item assignment

---

## Tipos de datos básicos 🔢

Los tipos de datos básicos de `Python` también son `inmutables`. Un `número entero`, un `número decimal` o un valor `booleano` no pueden modificarse una vez creados en memoria.

Lo cual es muy interesante, ¿cómo pueden ser `inmutables` si podemos sumar dos valores `int` y guardar el resultado en una variable, sumarle a un `float` o asignar un nuevo `bool`?

Cuando escribimos algo como `score = score + 10`, no estamos modificando el objeto que contiene el valor anterior. `Python` crea un **objeto nuevo** con el resultado de la operación y hace que la variable apunte a ese nuevo objeto.

Podemos comprobarlo usando la función `id()`, que devuelve el identificador único del objeto en memoria. Si el `id` cambia después de una operación, significa que estamos ante un objeto nuevo, no el mismo modificado.

```python
score = 100
print("ID antes:", id(score))  # Identificador del objeto en memoria

score = score + 10
print("ID despues:", id(score))  # Nuevo objeto, nuevo identificador
print("Valor:", score)
```

Copiar

✅ ID antes: 140720313429576
ID despues: 140720313429896
Valor: 110

💡 El `id` cambió porque `score` ahora apunta a un objeto nuevo. El entero `100` no fue modificado, simplemente dejó de ser referenciado por `score`.

---

## Conclusión 🎉

Hoy conocimos una de las características más fundamentales del sistema de tipos de `Python`, la `mutabilidad`. La cual existe porque todos los datos tienen el mismo rol en un programa, y `Python` nos da herramientas para manejar y expresar esa diferencia.

La elección entre uno y otro es una forma de **expresar nuestra intención** como programadores.

Cada vez que declaramos una variable en `Python`, estamos tomando una decisión. Ahora sabemos cómo tomarla con criterio.